Projet MLSF : résolution du jeu Meowdoku

Question 4 : heuristiques de branchement (DPLL_H)

1 Objectif et organisation

L'algorithme 3 de l'énoncé est l'algorithme 2 dans lequel la fonction `RandomChoice` (ligne 17) est remplacée par `Heuristics(F, H)`. La question demande d'implémenter au moins quatre heuristiques H, dont au moins une fondée sur le choix d'un littéral, une qui tient compte de la taille des clauses et une dynamique, puis de vérifier qu'elles donnent les mêmes résultats que l'algorithme 2.

Nous avons implémenté **toutes** les heuristiques de l'énoncé, dans leurs deux variantes statique et dynamique, ainsi qu'une heuristique de notre choix (H8), soit 16 heuristiques :

Nom	Heuristique	Sélection
`H0`	CA : choix aléatoire (algorithme 2)	variable
`H1`	PVL : première variable libre, valeur 1	variable
`H2s`, `H2d`	VPPM : variable la plus fréquente	variable
`H3s`, `H3d`	LP : littéral le plus fréquent	littéral
`H4s`, `H4d`	VFCM (MOMS) : clauses de taille minimale	variable
`H5s`, `H5d`	LFCM : clauses de taille minimale	littéral
`H6s`, `H6d`	VJW : poids de Jeroslow-Wang	variable
`H7s`, `H7d`	LJW : poids de Jeroslow-Wang	littéral
`H8s`, `H8d`	JWP : Jeroslow-Wang, littéraux positifs (section 7)	littéral

Le suffixe `s` désigne la variante statique, `d` la variante dynamique. Les contraintes de l'énoncé sont donc couvertes plusieurs fois : choix d'un littéral (H3, H5, H7, H8), taille des clauses (H4 à H8), variantes dynamiques (H2d à H8d).

Le code se répartit ainsi :

— `heuristics.py` (nouveau) : les heuristiques et la fonction `dpll_h(clauses, heuristic)` qui implémente l'algorithme 3 ;

— `dpll.py` : la fonction de branchement de `dpll` devient un paramètre, et un compteur du temps passé à choisir est ajouté (préparation de la question 5) ;

— `tests/test_heuristics.py` (nouveau) : tests unitaires ;

— `validate.py` : option `-heuristics` pour comparer les heuristiques à l'algorithme 2 sur des fichiers DIMACS.

2 Intégration dans DPLL : une seule implémentation

Choix. Plutôt que d'écrire une seconde fonction DPLL, la fonction `dpll` de la question 2 reçoit un paramètre facultatif `choose`, une fonction qui prend la formule courante et renvoie un littéral :

```
def dpll(clauses, rng=None, stats=None, choose=None):
    if choose is None:
        rng = rng if rng is not None else random.Random()
        choose = lambda current: random_choice(current, rng)
    ...
```

```
        literal = choose(clauses)      # ligne 17 de l'algorithme 3
```

Comme à la question 2, le couple (p, b) est codé par un seul littéral : p si $b = 1$, $-p$ si $b = 0$; la branche « sinon » utilise son opposé. `heuristics.dpll_h` construit la fonction `choose` correspondant au nom de l'heuristique et appelle `dpll`.

Justification. L'algorithme 3 ne diffère de l'algorithme 2 que par la ligne 17 : partager le code garantit que toutes les autres étapes (clause vide, clause unitaire, littéral pur, construction de la valuation) sont exactement celles qui ont été validées à la question 2. Une copie de `dpll` aurait pu diverger lors d'une correction future. Sans paramètre `choose`, le comportement de la question 2 est inchangé ; un test vérifie qu'avec la même graine, `H0` donne exactement le même résultat et le même nombre de décisions que l'algorithme 2.

Fonctions plutôt que classes. Une heuristique est une simple fonction `choose(clauses)`. Les variantes statiques ont besoin d'un état (l'ordre calculé au départ) : il est capturé par une fermeture (fonction définie à l'intérieur de `make_static`). Cela évite de définir une hiérarchie de classes pour un comportement d'une ligne.

3 Ce que voit l'heuristique

La formule n'a que des clauses de taille au moins 2. La ligne 17 n'est atteinte que si la formule courante ne contient ni clause vide, ni clause unitaire, ni littéral pur, et n'est pas vide. Elle contient donc au moins une variable, et $k = \min_{C \in F} |C| \geq 2$. Les heuristiques n'ont pas à traiter de cas particulier.

« Variable non affectée » = variable présente dans la formule courante. Notre DPLL ne stocke pas d'affectation pendant la descente : il simplifie la formule (`simplify`). Une variable absente de la formule courante est soit déjà affectée, soit apparue seulement dans des clauses déjà satisfaites ; dans les deux cas la brancher serait inutile (la formule ne changerait pas). Les heuristiques choisissent donc parmi les variables de la formule courante, comme `random_choice` à la question 2. C'est aussi ce qui rend H1 (PVL) correcte : c'est la plus petite variable de la formule courante.

4 Une structure commune à H2–H8

Toutes les heuristiques H2 à H8 comptent des occurrences de littéraux, chaque occurrence étant pondérée par un poids qui ne dépend que de la taille de sa clause. Elles ne diffèrent que par ce poids et par le mode de sélection :

Heuristiques	Poids d'une clause C	Fonction
H2, H3	1	`weight_one`
H4, H5	1 si $\|C\| = k$, 0 sinon	`weight_min_size`
H6, H7, H8	$2^{-\|C\|}$	`weight_jeroslow_wang`

Une seule fonction calcule ces scores :

```
def literal_scores(clauses, weight):
    min_size = min((len(clause) for clause in clauses), default=0)
    scores = Counter()
    for clause in clauses:
        w = weight(len(clause), min_size)
        if w:
```

```
            for literal in clause:
                scores[literal] += w
    return scores
```

Avec le poids 1, `scores[l]` vaut $occ(l, F)$, et le score d'une variable est `scores[p]+scores[-p]` $= \#(p, F)$. Avec le poids de H4, on obtient $occ_k(l, F)$ et $\#_k(p, F)$; avec $2^{-|C|}$, les sommes de Jeroslow-Wang $\sum_k occ_k(l, F)\, 2^{-k}$ et $\sum_k \#_k(p, F)\, 2^{-k}$.

Deux modes de sélection, décrits par une clé de tri. Le littéral choisi est celui qui maximise une clé :

- **variable** (H2, H4, H6) : la clé de $\pm p$ est $(\#(p),\ -p,\ \pm p$ est la polarité majoritaire$)$; on obtient la variable de plus grand score avec sa polarité majoritaire, 1 si $\#^+ \geq \#^-$ comme dans l'énoncé ;
- **littéral** (H3, H5, H7) : la clé de l est $($`scores[l]`$,\ -|l|,\ l > 0)$.

Égalités. En cas d'égalité de score, la plus petite variable l'emporte, puis la polarité positive (le $-p$ et le $l > 0$ des clés). Le choix est donc déterministe : une exécution ne dépend pas de l'ordre interne des `set` Python et peut être reproduite exactement, ce qui facilite le débogage et les mesures. Seule H0 utilise le hasard (avec une graine).

Justification. Factoriser les six heuristiques de l'énoncé en « poids + sélection » tient en 200 lignes commentaires compris, met en évidence ce qui les distingue réellement et garantit qu'un même calcul de scores est utilisé partout (un bug ne peut pas toucher une seule heuristique sans être visible sur les autres). Ajouter une heuristique (H8) a demandé trois lignes.

5 Variantes statiques et dynamiques

Dynamique. Les scores sont recalculés sur la formule courante à chaque décision, ce qui coûte un parcours de la formule ($O(|F|)$, le même ordre que `simplify`) :

```
def choose(clauses):
    key = make_key(literal_scores(clauses, weight))
    return max(literals_of(variables_of(clauses)), key=key)
```

Statique. Comme le suggère l'énoncé, un prétraitement calcule les scores une seule fois sur la formule initiale et trie les littéraux par clé décroissante ; on ne garde que le premier littéral de chaque variable. Choisir revient ensuite à prendre le premier littéral de cet ordre dont la variable est encore présente dans la formule courante. Sur la formule initiale, la variante statique fait donc exactement le même choix que la variante dynamique (c'est testé) ; elles divergent ensuite, quand la formule a été simplifiée.

Le choix statique parcourt encore la formule pour connaître ses variables : nous l'avons accepté pour garder la représentation de la question 2 (une liste de clauses) sans structure supplémentaire. Ce parcours est bien moins coûteux que le calcul des scores, et la question 6 pourra le supprimer avec une affectation explicite.

H4 statique. Pour la variante statique, k est la taille minimale des clauses de la formule initiale ; les variables qui n'apparaissent dans aucune clause de taille k ont un score nul et sont rangées après les autres, par numéro croissant.

6 Vérification sur l'exemple de l'énoncé

Pour $F = (x_1 \vee x_2 \vee \neg x_4) \wedge (x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_4 \vee \neg x_5)$, les scores calculés à la main sont :

	x_1	x_2	x_3	x_4	x_5
$\#^+/\#^-$	2 / 0	2 / 1	1 / 0	1 / 1	0 / 1
$\#_2$ $(k=2)$	0	1	1	0	0
$\sum_k \#_k\, 2^{-k}$	3/16	7/16	1/4	3/16	1/16

D'où les choix : H1 $\to x_1$; H2 $\to x_2 = 1$ ($\#(x_2) = 3$, maximum) ; H3 $\to x_1$ ($occ(x_1) = occ(x_2) = 2$, égalité départagée par le numéro) ; H4 et H5 $\to x_2 = 1$ (x_2 et x_3 à égalité dans la seule clause de taille 2) ; H6 $\to x_2 = 1$ (poids 7/16, la valeur donnée dans l'énoncé) ; H7 et H8 $\to x_2$ (poids de x_2 : $1/8 + 1/4 = 3/8$). Ces choix sont vérifiés par `test_choix_sur_l_exemple`.

Lecture de l'énoncé. Quelques coquilles ont été interprétées :

- pour H5, l'énoncé indique « 1 si l est de la forme $\neg p$ » dans la première phrase, puis 0 dans la définition formelle : nous affectons la valeur qui rend l vrai (0 pour $\neg p$), seule lecture cohérente avec H3 et H7 ;
- pour H7, la formule écrit $\#_k(l, F)$; nous utilisons $occ_k(l, F)$, comme dans la phrase qui la précède ;
- dans l'exemple, la ligne « $\#_2(x_2, F) = 1$, $\#^+(x_2, F) = 2$ » mélange les notations ; nos valeurs de $\#_2^+$ et $\#_2^-$ sont 1 et 0.

7 Heuristique ajoutée : H8, Jeroslow-Wang positif

Constat. Les premières mesures (section 9) montrent que, sur les grilles Meowdoku, toutes les heuristiques H2 à H7 font *plus* de décisions que le choix aléatoire. L'explication vient de l'encodage de la question 1 : les contraintes « au plus un chat » (par ligne, colonne, région et voisinage) donnent un très grand nombre de clauses binaires $\neg a \vee \neg b$, toutes négatives ; seules les clauses « au moins un chat par région » contiennent des littéraux positifs, et elles sont longues. Toute heuristique fondée sur la fréquence ou la polarité majoritaire choisit donc presque toujours « pas de chat dans cette case ». Cette décision ne fait disparaître que quelques clauses et ne déclenche presque aucune propagation. À l'inverse, placer un chat (affecter une variable à 1) rend unitaires toutes les clauses binaires de la case : la propagation élimine toute la ligne, la colonne, la région et les voisins.

Définition. H8 utilise les poids de Jeroslow-Wang (comme H7) mais ne choisit que parmi les littéraux *positifs* présents dans la formule ; s'il n'en reste aucun, elle se comporte comme H7. Sa clé est ($l > 0$ et $occ(l) > 0$, `scores[l]`, $-|l|$, $l > 0$). Sur Meowdoku, un littéral positif n'apparaît que dans la clause « au moins un chat » de sa région, de taille le nombre de cases encore libres de la région ; son poids est $2^{-\text{taille}}$. H8 place donc un chat dans la **région qui a le moins de cases possibles**. C'est le principe classique « variable la plus contrainte d'abord » des problèmes de contraintes : si la région n'a que deux cases libres, l'une des deux branches est la bonne et l'échec est détecté vite.

Pourquoi la garder générale. H8 est définie pour toute formule CNF, pas seulement pour Meowdoku : elle passe les mêmes tests que les autres heuristiques et reste raisonnable sur les instances aléatoires (voir les mesures). Elle est la candidate naturelle pour le solveur optimisé de la question 6.

8 Validation

L'énoncé demande que les heuristiques donnent des résultats conformes à l'algorithme 2 sur les données de test de la question 2. Trois niveaux de vérification ont été mis en place.

Tests unitaires (`tests/test_heuristics.py`).

- les comptages et le poids 7/16 de l'exemple de l'énoncé, et le choix de chaque heuristique sur cet exemple ;
- la polarité minoritaire n'est jamais choisie par VPPM ; H8 préfère un littéral positif à un négatif de meilleur poids ;
- statique et dynamique coïncident sur la formule initiale et divergent après une simplification (exemple explicite) ;
- une variante statique ne renvoie jamais une variable sortie de la formule courante ;
- pour chacune des 16 heuristiques : cas limites (formule vide, clause vide, tautologie, exemple des lampes), 400 formules aléatoires comparées à une force brute qui essaie les 2^n valuations, grilles Meowdoku SAT de taille 4 à 8 et une grille UNSAT ; le modèle renvoyé est vérifié clause par clause ;
- `H0` reproduit exactement l'algorithme 2 à graine égale.

Tous les tests passent :

```
uv run python -m unittest discover -s tests -t .
```

Instances SATLIB et instances publiques. L'option `-heuristics` de `validate.py` lance, en plus des algorithmes 1 et 2, chaque heuristique demandée. Elle doit renvoyer le même statut que l'algorithme 2 (et que le statut attendu) et, si SAT, un modèle qui satisfait toutes les clauses. Une heuristique qui dépasse le délai n'est pas une erreur mais elle est signalée.

```
uv run python validate.py data/satlib --heuristics all --timeout 10
uv run python validate.py public-instances --heuristics all --timeout 20
```

Si l'algorithme 2 dépasse lui-même le délai, chaque heuristique est comparée au statut attendu (fichier `expected.json` ou nom du fichier SATLIB).

Résultats.

- **SATLIB** (3200 instances uf20 à uf75 et uuf50 à uuf75 : 2100 SAT, 1100 UNSAT) : les 16 heuristiques donnent le statut attendu sur toutes les instances, avec un modèle correct pour chaque instance SAT, aucun dépassement du délai de 10 s.
- **Instances publiques** (41 fichiers, délai de 20 s) : aucune erreur. Les 6 cas limites `edge_*` et les tests d'interface passent pour toutes les heuristiques. L'algorithme 2 dépasse le délai sur public-03, public-11, public-12 et public-27 ; plusieurs heuristiques les résolvent (11 heuristiques sur 16 pour public-11, 12 pour public-27, toutes les variantes dynamiques pour public-12). Seule public-03, l'une des deux instances annoncées comme difficiles, n'est résolue par aucune. À l'inverse, quelques heuristiques dépassent le délai là où l'algorithme 2 conclut (H2 sur public-04 et public-21, H8s sur public-04, H3d, H5d, H7 et H8s sur public-06) : une heuristique n'est pas meilleure que le hasard sur toutes les instances.

9 Premières mesures

L'analyse détaillée des performances est l'objet de la question 5. Pour justifier nos choix, nous avons tout de même comparé les heuristiques sur trois familles : 40 instances SATLIB uf50 (SAT), 40 instances uuf50 (UNSAT) et 12 grilles Meowdoku générées de taille 8, 10 et 12 (délai de 10 s, aucun dépassement). Le tableau donne la médiane du nombre de décisions et la part du temps total passée dans le choix de branchement (`Stats.heuristic_time`).

Heuristique	uf50 (SAT) décisions	uf50 (SAT) % heur.	uuf50 (UNSAT) décisions	uuf50 (UNSAT) % heur.	Meowdoku décisions	Meowdoku % heur.
H0 (hasard)	160	10	432	9	14	7
H1	190	10	436	9	14	7
H2s / H2d	29 / 20	10 / 40	74 / 55	8 / 35	62 / 82	19 / 70
H3s / H3d	31 / 20	10 / 40	101 / 84	9 / 33	62 / 82	19 / 68
H4s / H4d	29 / **12**	10 / 34	74 / **32**	8 / 27	52 / 78	16 / 70
H5s / H5d	31 / 20	10 / 30	101 / 52	9 / 24	52 / 82	16 / 68
H6s / H6d	29 / 14	10 / 41	74 / 35	8 / 34	59 / 76	22 / 72
H7s / H7d	31 / 16	10 / 43	101 / 55	9 / 34	62 / 82	18 / 70
H8s / H8d	22 / 18	11 / 44	102 / 67	9 / 37	**7** / 8	5 / 27

Observations.

— Sur le 3-SAT aléatoire, toute heuristique fondée sur les occurrences divise le nombre de décisions par 5 à 13 par rapport au hasard, et le temps total par 5 à 11 (de 1,9 s à 0,17 s pour les 40 instances uf50 avec H4d). H1 n'apporte rien : l'ordre des numéros de variables n'a pas de sens sur une formule aléatoire.

— Les variantes statiques H2s, H4s et H6s sont *identiques* sur ces instances (de même que H3s, H5s et H7s) : toutes les clauses initiales ont 3 littéraux, donc les trois pondérations classent les variables de la même façon. Les différences n'apparaissent qu'avec les variantes dynamiques, quand les clauses raccourcissent.

— Le dynamique réduit encore le nombre de décisions (jusqu'à un facteur 2,5 pour H4 sur uuf50) mais 25 à 45 % du temps est alors passé à recalculer les scores, contre 10 % en statique. Sur ces petites instances le gain en temps reste faible ; sur l'instance publique public-12, en revanche, *toutes* les variantes dynamiques concluent en moins de 20 s et *aucune* variante statique. Le compromis « coût de la mise à jour / pertinence » de l'énoncé sera étudié à la question 5.

— Sur Meowdoku, H2 à H7 font 4 à 6 fois *plus* de décisions que le hasard, pour la raison exposée en section 7. H8 divise par deux le nombre de décisions du hasard, tout en restant proche des meilleures heuristiques sur SATLIB.

Ces mesures confirment l'intérêt d'avoir implémenté toutes les variantes plutôt que le minimum demandé : aucune heuristique n'est la meilleure partout, et le classement dépend fortement de la structure des instances.